# PPSITMAJM402 DEEP LEARNING

Ms. Sophia Dcruz

# UNIT I: APPLIED MATH AND MACHINE LEARNING BASICS

## CHAPTER 1: LINEAR ALGEBRA

# LINEAR ALGEBRA

- Linear Algebra is a widely used branch of mathematics
- Related with mathematical structures which are **continuous rather than discrete mathematics**
- Includes operations like:
  - Addition
  - Scalar multiplication
- Helps to understand concepts such as:
  - Linear transformations
  - Vector spaces
  - Linear equations
  - Matrices
  - Determinants
- A good knowledge of Linear Algebra is important for:
  - Machine Learning algorithms
  - Deep Learning algorithms

# Scalars

- A scalar is a **single number**
- Represents **0th order tensor**
- Written in **lowercase and italics**
- Example: $n$
- Scalars belong to different sets:
  - **R** → Real numbers
  - **N** → Positive integers (1, 2, 3, …)
  - **Z** → Integers (positive, negative, zero)
  - **Q** → Rational numbers
- **Example:**
- 5, –3, 2.7 are scalars

# Vector

- A vector is an **ordered array of single numbers**
- Represents **1st order tensor**
- Written in **lowercase, bold and italics**
- Representation: $x = [x_1\ x_2\ x_3\ \ldots\ x_{\square}]$
- Scalar element at ith position is written as: x[i]
- **Example:**
- $x = [2, 4, 6]$
- $x_1 = 2$, $x_2 = 4$, $x_3 = 6$
- **Special notation:**
- $x_{-1} \rightarrow$ vector consisting of all elements except $x_1$
- $x_{-\square} \rightarrow$ vector consisting of all elements except elements in set S
- **Vector Space:** A collection of all possible vectors of a particular dimension

# Matrix

- A matrix is a **2-D rectangular array of numbers**
- Represents **2nd order tensor**
- Written in **uppercase, bold and italics**
- If $p, q \in N$: $p \times q$ matrix has p rows and q columns
- **Example:** A $2 \times 2$ matrix

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

# Matrix Operations

- **Matrix Addition:** Possible only when **shapes of matrices are equal**
- **Example:**

[1 1] + [2 2] = [3 3]

[3 3]  [4 4]  [7 7]

- **Matrix–Scalar Addition:** Scalar is added to **each element of matrix**
- **Example:**

[1 1] + 1 = [2 2]

[3 3]  [4 4]

- **Matrix–Scalar Multiplication:** Scalar multiplies **each element of matrix**
- Example:

$$\begin{bmatrix} 2 & 3 \\ 3 & 4 \end{bmatrix} \times 3 = \begin{bmatrix} 6 & 9 \\ 9 & 12 \end{bmatrix}$$

- **Matrix Transpose:**

- **Matrix Transpose**
- Converts:
  - Row vector → Column vector
  - Column vector → Row vector
- Denoted as $\mathbf{A^T}$
- Property: $(A^T)_{i\square} = A_{\square i}$
- Example:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

# Tensors

- An array with **more than 2 axes**
- Wraps:
  - Scalar
  - Vector
  - Matrix
- Elements accessed using indices (i, j, k)
- **Example:**
- Image data (height × width × color channels)

# Multiplying Matrices and Vectors

- Let:
  - A be $m \times n$ matrix
  - B be $k \times p$ matrix
- Matrix multiplication AB is possible **only if $n = k$**
- Resulting matrix size: **$m \times p$**
- **Rule:**
- Number of columns in A = Number of rows in B

# Dot Product

- Combines corresponding elements and sums them
- **Example:**

$[1\ 2\ 3] \cdot [4\ 5\ 6] = 1\times4 + 2\times5 + 3\times6 = 32$

# Properties of Matrix Multiplication

- **Transpose Property:** $(AB)^T = B^T A^T$
- **Not Commutative:** $AB \neq BA$
- **Associative:** $A(BC) = (AB)C$
- **Distributive**
  - $A(B + C) = AB + AC$
  - $(A + B)C = AC + BC$

# Identity Matrices

- Square matrix
- Diagonal elements = 1
- All other elements = 0
- Also called:
  - Unit Matrix
  - Elementary Matrix
- Denoted as **I**
- **Property:**
- IA = A
- AI = A
- **Example:**

$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

# Inverse Matrix

- Exists only for **square matrices (**Same number of rows and columns)
- Inverse of A is denoted as $\mathbf{A^{-1}}$
- Property:
  - $AA^{-1} = A^{-1}A = I$
- If inverse exists:
  - Matrix is **invertible / non-singular**
- Non-square matrices do **not** have inverses

# Linear Dependence and Span

- Consider equation: $Ax = b$
- System can have:
  - One solution
  - No solution
  - Infinitely many solutions
- Columns of A represent **directions from origin**
- Determines whether vector b can be reached

# Span

- **Linear Combination:** Obtained by multiplying vectors with scalars and adding them
- **Example:** $\mathbf{v = 2v_1 + 3v_2}$
- Set of all vectors formed by linear combinations
- Span of columns of A is called:
  - Column Space
  - Range of A
- **Example:** If vectors lie on a plane, their span is the plane

# Linear Independence

- A set of vectors is linearly independent if:
- Only trivial solution exists
- Equation: $a_1v_1 + a_2v_2 + \ldots + a_nv_n = 0$
- Trivial solution: all scalars = 0
- **Linear Dependence**
- If non-trivial solution exists
- One vector can be expressed using others
- **Example:** $v_3 = v_1 + v_2 \rightarrow$ linearly dependent

# Norms

- Norm is a **function that measures length/size of any vector** (excluding zero vectors)
- In machine learning, size of vectors is frequently measured using **norms**
- Norms belong to **p-norms or (lp-norms) family**
- Here **p ≥ 1**

# p-Norm of a Vector

- The p-norm of vector **x** can be presented as:
  - Each element of vector **x** is raised to power **p**
  - All values are summed
  - The sum is raised to power **1/p**
- Simply represented as **lp-norm**
- **Example:**
- For x = [1, 2, 3] and p = 2 → Euclidean norm

# Properties of a Norm Function

- Any function **f** is a norm if it satisfies:
- **Positive Definiteness**
  - If $f(x) > 0$ then $x \neq 0$ (Zero Vector)
  - If $f(x) = 0$ then $x = 0$
- **Scalar Multiplication Property**
  - For any scalar K: $f(Kx) = K\, f(x)$
- **Triangle Inequality**
  - For any vector y: $f(x + y) \leq f(x) + f(y)$
- If all three properties are satisfied, then **f is a norm**

# Manhattan Distance (1-norm)

- Also called **1-norm**
- Measures distance between two points that can travel along **orthogonal blocks**
- **Example:**
- x = [2, 4]
- 1-norm = |2| + |4| = 6
- **Application:**
- Used in optimization and sparse models

# Infinity-Norm (Max-Norm)

- Also called **max-norm**
- Returns the **maximum absolute value** in the vector
- **Example:**
- $a = [2, 4, -5]$
- $\infty$-norm = 5

# Euclidean Norm (2-norm)

- Most commonly used norm
- Used to calculate **magnitude of a vector**
- Also called **L2-norm**
- **Example:**
- $x = [3, 4]$
- 2-norm $= \sqrt{(3^2 + 4^2)} = \sqrt{(9 + 16)} = \sqrt{25} = 5$
- **Application:**
- Distance calculation, machine learning algorithms

# Special Kinds of Matrices and Vectors

- **Diagonal Matrix**
- A diagonal matrix is denoted by **D**
- Diagonal elements are **non-zero**
- All other elements are **zero**
- **Example:**
- D = [1 0 0

  0 2 0

  0 0 3]
- As a vector: $d = (d_{11}, d_{22}, d_{33})$
- With scalar values: d = (1, 2, 3)
- Diagonal matrix is **not compulsory square**
- Rectangular diagonal matrices are also possible

**Triangular Matrix**

- A triangular matrix is a **square matrix**
- Elements are filled either:
  - Above the main diagonal
  - Below the main diagonal
- Remaining elements are **zero**

**Upper Triangular Matrix**

- Values lie **above the main diagonal**
- **Example:**

[1 2 3

0 4 5

0 0 6]

**Lower Triangular Matrix**

- Values lie **below the main diagonal**
- **Example:**

[1 0 0

2 3 0

4 5 6]

**Symmetric Matrix**

- A matrix where: Top-right triangle = Bottom-left triangle
- Transpose of symmetric matrix equals original matrix
- **Property:**
- $A = A^T$
- **Example:**

[1 2 3

2 4 5

3 5 6]

**Orthogonal Matrix**

- A matrix is orthogonal if: Dot product of two vectors = 0
- It is a **square matrix**
- Rows and columns are:
  - Orthonormal
  - Unit vectors
- Vectors are:
  - Perpendicular
  - Magnitude = 1
- **Property:**
- $A^T A = A A^T = I$
- **Example:**

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

# Eigen Decomposition

- We usually used to see multiple **mathematical structures which are complex to understand**
- Hence, there is a need to **break complex structures into simplified form**
- Just like integers can be decomposed into **prime factors**, a matrix can also be decomposed
- **Matrix decomposition** helps to:
  - Simplify complex operations
  - Understand useful properties of a matrix
- Matrix decomposition tools help in **reducing a matrix to its constituent parts**

- Eigen decomposition is a **widely used matrix decomposition**
- It involves decomposition of a **square matrix** into:
  - Eigenvectors
  - Eigenvalues
- This decomposition is very useful in **machine learning**
- Important application:
  - **Principal Component Analysis (PCA)**

# Eigenvector Definition

- A vector is an **eigenvector** of a matrix if it satisfies:
- $Av = \lambda v$
- Where:
  - **A** → Parent square matrix
  - **v** → Non-zero eigenvector
  - **λ (lambda)** → Eigenvalue (scalar)
- **Example (Conceptual):**
- After multiplying matrix A with vector v, the direction of v remains same, only its magnitude changes by λ

# Eigenvalue Equation

- The equation $\mathbf{Av} = \lambda\mathbf{v}$ is called the **eigenvalue equation**
- It defines the relationship between:
  - Matrix
  - Eigenvector
  - Eigenvalue

# Matrix Representation Using Eigen Decomposition

- The parent matrix can be expressed as:
- $A = V\, diag(\lambda)\, V^{-1}$
- Where:
  - **V** → Matrix comprised of eigenvectors
  - **diag(λ)** → Diagonal matrix comprised of eigenvalues
  - $\mathbf{V^{-1}}$ → Inverse of the eigenvector matrix

# YouTube Reference for Eigen Value and Eigen Vector

- https://youtu.be/h8sg_XBp6VA?si=oJS8oWYp7nAdQYqS
- https://youtu.be/C21DPLcmCTE?si=E8M9jEArQltb9ABF

# UNIT I: APPLIED MATH AND MACHINE LEARNING BASICS

## CHAPTER 2: NUMERICAL COMPUTATION

# INTRODUCTION TO NUMERICAL COMPUTATION

- A lot amount of **computation is needed in machine learning algorithms**
- Used to solve **complex and vital problems**
- Requires an **iterative process**
- Mathematical methods are used repeatedly to get **best solutions**
- Numerical computation involves a **large number of arithmetic calculations**
- Hence, requires **efficient and fast computational devices**
- The approach:
  - Formulate a **mathematical model**
  - Solve problems using **arithmetic calculations**
- **Example:** Training a neural network involves repeated numerical calculations of weights and gradients

# OVERFLOW AND UNDERFLOW

- One major difficulty in computers is: Representing **infinitely many real numbers** using **finite memory**
- Every calculation introduces **approximation error**
- One common error: **Rounding error**
- Algorithms may fail if **rounding errors accumulate**
- Numbers in computers are stored as **discrete digits**
- Arithmetic operations may produce:
  - Extra digits that cannot be stored
  - Leading to **overflow or underflow**

# Underflow

- Underflow occurs when: Result of a mathematical operation is **smaller than the device can store**
- Happens mainly with: **Floating-point precision**
- Due to discrete nature of storage: Very small numbers cannot be represented
- Numbers close to **0 are rounded to 0**
- **Example:**
- $0.005 \times 0.005 = 0.000025$
- Stored as 0.000 due to limited precision
- This is an **erroneous answer**
- Underflow is a **representational error**
- Occurs mostly in **decimal arithmetic**
- Can also occur when two negative numbers are added and result is out of range

# Overflow

- Overflow occurs when: Result of a calculation is **too large to be represented**
- Caused due to: **Limited memory size**
- In mathematics: Numbers are absolute
- In computers: Numbers are rounded due to **hardware limitations**
- **Example:**
- Maximum number in 1 byte = 255
- Adding 2 results in 257 (requires 9 bits)
- System can store only 8 bits → **overflow**

# Softmax Function and Numerical Stability

- One important function affected by overflow and underflow: **Softmax function**
- Used to predict probabilities for a **Multinoulli Distribution**
- If input is:
  - Very large positive → **overflow**, softmax returns NaN
  - Very large negative → **underflow**, softmax becomes zero
- Leads to: **Division by zero**

# Stabilization Technique

- Solution to overflow and underflow: **Stabilization**
- Define a new vector: Subtract the largest element from all elements
- Results:
  - At least one element becomes zero
  - No overflow
  - No underflow
  - No division by zero
- This process is called **stabilization**
- Used to solve numerical problems **analytically**

# POOR CONDITIONING

- Conditioning refers to: How much a function changes due to **small changes in inputs**
- Functions that change rapidly are: **Problematic for scientific computation**
- Small rounding errors in input can cause: **Large changes in output**

# Condition Number

- Consider a function on vector x
- If matrix has eigen decomposition
- Condition number is defined as: Ratio of **largest eigenvalue** to **smallest eigenvalue**
- When this ratio is large: Matrix inversion becomes **sensitive to errors**
- **Example:**
- Ill-conditioned matrices lead to unstable solutions in ML models

# GRADIENT BASED OPTIMIZATION

- Most machine learning and deep learning algorithms involve **optimization**
- Optimization means: Minimizing or maximizing a function f(x)
- Function to be optimized is called:
  - **Objective Function,** also known as **Criterion**

**Optimization Notation**

- Optimal value denoted by superscript *
- Single variable: x* = arg min f(x)
- Multi-variable: x* = arg min f(x)
- Maximization can be converted to: Minimization of −f(x)

**Objective Function Terminology**

- Objective function is also called:
  - Loss function
  - Cost function
  - Error function
- Example: Linear least squares

# Gradient Descent Concept

- Derivative of function $f(x)$: $f'(x)$ or $dy/dx$
- Derivative represents: Slope of function
- Shows how small change in input affects output
- Reducing $f(x)$: Move $x$ in opposite direction of derivative
- This technique is called **Gradient Descent**
- For $x > 0$:
  - $f(x)$ increases
  - $f'(x) > 0$
- For $x < 0$:
  - $f(x)$ decreases
  - $f'(x) < 0$
- Always move in Opposite direction of derivative

# Stationary Points and Local Optima

- When $f'(x) = 0$: No direction information
- Such points are called **Stationary or Critical Points**
- Types:
- Local minimum
- Local maximum
- Saddle point (neither max nor min)
- Optimization algorithms may: Fail to find global minimum
- Usually accept local minimum solutions

# CONSTRAINT OPTIMIZATION

- Sometimes optimization is required with constraints
- Solution x must lie in Set S
- Such solutions are called **Feasible solutions**

**Constraint Examples**

- Constraint like: $||x|| \leq 1$
- Simple approach:
  - Modify gradient descent using **Lagrangian formulation**

# Lagrangian Method

- Used for:
  - Equality constraints
- Converts constrained problem into Unconstrained optimization problem
- Least squares with constraint:
  - Introduce Lagrangian
  - Solve by differentiation